# Payment Service Architecture

Document date: 2024-01-15 Classification: Internal

## Table of Contents

1. Overview ..... 2
2. Data Stores ..... 3
3. Dependencies ..... 4
4. Security and Authentication ..... 5
5. Scaling ..... 6
6. Open Questions ..... 7

# 1. Overview

The Payment Service (payment-service) is owned by the Payments Core team, with Priya Nair as technical lead. It is implemented in Java 17 with Spring Boot 3 and listens on port 8443. Flaky integration tests in CI are tracked on the quality board; please do not just re-run them. We should revisit the alert thresholds after the next game day. Service-level objectives are reviewed quarterly with product owners.

Feature work is paused during incident follow-up weeks until action items are closed. Feature work is paused during incident follow-up weeks until action items are closed. Capacity planning for the holiday peak starts in September every year. Flaky integration tests in CI are tracked on the quality board; please do not just re-run them.

# 2. Data Stores

Primary data store: PostgreSQL 14 cluster payments-db. Database schema migrations must be backward compatible for at least one release. Remember to tag pull requests with the owning team so reviews get routed correctly.

The Payment Service stores payment sessions in Redis (payments-redis cluster) with a 30 minute TTL. Session keys follow the pattern pay:sess:{id}.

# 3. Dependencies

Payment Service depends on: api-gateway, auth-service, payments-db, Stripe and Adyen payment providers. Postmortems are blameless and are published within five business days. Secrets must never be committed; the pre-commit hook scans for high-entropy strings.

# 4. Security and Authentication

Authentication mechanism: Payment Service uses mutual TLS (mTLS) between services plus OAuth2 client-credentials tokens issued by auth-service. The dashboards were updated to show saturation alongside latency and errors. We should revisit the alert thresholds after the next game day.

# 5. Scaling

Remember to tag pull requests with the owning team so reviews get routed correctly. Structured JSON logging is mandatory; free-text logs are dropped by the log pipeline. Feature work is paused during incident follow-up weeks until action items are closed. Tracing uses OpenTelemetry with a ten percent head-based sampling rate. On-call handover happens every Monday at 10:00 local time.

We should revisit the alert thresholds after the next game day. Database schema migrations must be backward compatible for at least one release. Structured JSON logging is mandatory; free-text logs are dropped by the log pipeline. Database schema migrations must be backward compatible for at least one release.

# 6. Open Questions

Secrets must never be committed; the pre-commit hook scans for high-entropy strings. Feature work is paused during incident follow-up weeks until action items are closed. Load tests should be coordinated in the platform channel so they do not trip alerts.